

Diseño e Implementación de un Brazo Robótico de 3 GDL para Corte Láser No-Coplanar

Ivan Yerry Hinojosa Zegarra
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72241950@continental.edu.pe

Brayan Omar Duran Arroyo
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72542899@continental.edu.pe

Jorge Andre Tapia Patiño
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60761518@continental.edu.pe

Carlos Benedic Sosa Carhuamaca
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60847160@continental.edu.pe

Dick Jairo Guzman Zelada
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60195546@continental.edu.pe

Resumen—El presente artículo detalla el diseño, simulación y construcción física de un brazo manipulador antropomórfico de 3 grados de libertad (GDL) orientado al corte y perfilado láser no-coplanar. El objetivo principal ha sido desarrollar un sistema capaz de seguir trayectorias tridimensionales sobre superficies cilíndricas e irregulares mediante un emisor láser en su efector final, controlado por un microcontrolador ESP32. Se han implementado los modelos de cinemática directa, inversa y Jacobiana para garantizar el posicionamiento espacial en los tres ejes (X, Y, Z) y conservar la distancia focal del láser. La estructura física se fabricó en impresión 3D a partir de mallas STL, con un firmware desarrollado en C++. La validación cinemática se realizó mediante simulación virtual en ROS 2 y Gazebo de forma puramente offline.

Index Terms—Cinemática, brazo robótico, ESP32, C++, ROS 2, simulación virtual.

I. INTRODUCCIÓN

En la industria moderna, los brazos robóticos son herramientas fundamentales para tareas de precisión como la soldadura y el corte láser. Comprender la cinemática detrás de estos mecanismos es el primer paso para su control y programación [1]. Este proyecto busca aplicar los conceptos teóricos aprendidos en el curso de Fundamentos de Robótica —tales como matrices de transformación homogénea, representación de orientación y cinemática diferencial— en un caso de uso práctico.

Al estructurar el modelo con 3 grados de libertad (GDL) (rotación en la base, elevación en el hombro y elevación en el codo), se simplificó el análisis geométrico sin perder la capacidad de posicionar el efector final en un volumen de trabajo tridimensional. Además, el proyecto utiliza un emisor láser de baja potencia (Clase 1/2) como efector final. A diferencia de procesos de contacto como la soldadura por arco, el láser permite una validación cinemática de “no-contacto”. Esto simplifica el control al eliminar la necesidad de compensar fuerzas de fricción o colisiones mecánicas directas contra la pieza de trabajo, permitiendo un seguimiento de

trayectoria preciso y eficiente. Este enfoque es crucial para el corte y grabado sobre superficies irregulares y curvadas en tres dimensiones, donde el brazo debe seguir contornos tridimensionales curvos sobre una superficie no plana manteniendo constante la distancia focal del haz láser, una operación inviable para máquinas cartesianas 2D tradicionales. El desarrollo del software, simulación, firmware embebido e interfaz web se encuentra disponible de forma pública en el repositorio de GitHub del proyecto [2].

II. OBJETIVOS

A. Objetivo General

Desarrollar, simular y construir un brazo robótico articulado de 3 GDL orientado a procesos de corte láser no-coplanar sobre superficies curvas e irregulares, validando los modelos cinemáticos en simulación virtual (ROS 2/Gazebo) y, de forma independiente, en el prototipo físico controlado por un ESP32.

B. Objetivos Específicos

- Definir los parámetros de Denavit-Hartenberg y resolver analíticamente la cinemática directa, inversa y Jacobiana para garantizar el seguimiento espacial tridimensional (X, Y, Z) y la evasión de singularidades en superficies no planas.
- Modelar la estructura física en URDF/Xacro y simular el comportamiento del sistema utilizando el sistema operativo de robots (ROS 2, por sus siglas en inglés) y Gazebo de manera exclusivamente virtual, sin conexión física con el prototipo.
- Fabricar los eslabones y la base del robot utilizando tecnologías de impresión 3D a partir de archivos STL.
- Implementar el control de bajo nivel del hardware mediante un microcontrolador ESP32, utilizando C++ para accionar las articulaciones físicas del prototipo.

III. MODELADO CINEMÁTICO

Se utiliza la convención de Denavit-Hartenberg (D-H) clásica para describir la geometría y cinemática del brazo manipulador antropomórfico de 3 GDL.

A. Sistemas de Referencia y Parámetros D-H

Se definen cuatro sistemas de referencia locales $\{0\}$, $\{1\}$, $\{2\}$ y $\{3\}$ asociados a cada articulación móvil y al efector final de la estructura, coincidiendo con los marcos de simulación.

- $d_1 = 60,000$ mm: Altura del eslabón base fijo.
- $d_2 = 36,173$ mm: Desplazamiento vertical del hombro.
- $a_2 = 14,915$ mm: Desplazamiento radial del hombro (distancia horizontal entre el eje de rotación de la base y el eje del hombro).
- $a_3 = 146,190$ mm: Longitud del primer eslabón móvil (distancia entre hombro y codo).
- $a_4 = 160,823$ mm: Longitud plana del segundo eslabón móvil (distancia de codo a brida/láser).
- $d_4 = -4,000$ mm: Desplazamiento transversal (fuera de plano) del efector final.
- θ_1 : Ángulo de rotación de la base (alrededor del eje vertical Z_0).
- θ_2 : Ángulo del hombro.
- θ_3 : Ángulo del codo.

La convención clásica de D-H establece que la matriz de transformación homogénea entre eslabones consecutivos se calcula mediante:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

A partir de los sistemas de referencia y dimensiones físicas asignados, se obtiene la Tabla de Parámetros D-H (Tabla I). La Fig. 1 ilustra la disposición geométrica de estos parámetros sobre el modelo físico del brazo en su configuración de reposo ($\theta_1 = \theta_2 = \theta_3 = 0$).

Tabla I
PARÁMETROS DE DENAVIT-HARTENBERG REALES DEL PROTOTIPO

Eslabón (i)	θ_i	d_i (mm)	a_i (mm)	α_i
1 (0 $\rightarrow$ 1)	θ_1	$d_1 + d_2 = 96,173$	$a_2 = 14,915$	90°
2 (1 $\rightarrow$ 2)	θ_2	0	$a_3 = 146,190$	0
3 (2 $\rightarrow$ 3)	θ_3	$d_4 = -4,000$	$a_4 = 160,823$	0

B. Cinemática Directa

Sustituyendo los parámetros de la Tabla I en (1), se obtienen las matrices de transformación homogénea individuales para cada articulación y eslabón:

$$T_1^0 = \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & a_2 \cos \theta_1 \\ \sin \theta_1 & 0 & -\cos \theta_1 & a_2 \sin \theta_1 \\ 0 & 1 & 0 & d_1 + d_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

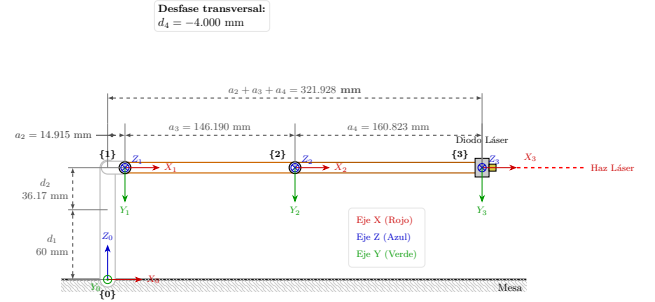


Figura 1. Disposición geométrica del brazo robótico láser en configuración de reposo ($\theta_1 = \theta_2 = \theta_3 = 0$). Se indican los sistemas de referencia D-H $\{0\}$ - $\{3\}$, las longitudes de eslabón a_3 , a_4 , los desplazamientos d_1 , d_2 y el desfase transversal d_4 .

$$T_2^1 = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & a_3 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & a_3 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$T_3^2 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & a_4 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & a_4 \sin \theta_3 \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

Multiplicando consecutivamente estas matrices se obtiene la matriz de transformación homogénea total del efector final respecto a la base del robot $T_3^0 = T_1^0 T_2^1 T_3^2$. Definimos esta matriz en función de las coordenadas del efector final $[x_E, y_E, z_E]^T$ dadas en (6)-(8):

$$T_3^0 = \begin{bmatrix} C_1 C_{23} & -C_1 S_{23} & S_1 & x_E \\ S_1 C_{23} & -S_1 S_{23} & -C_1 & y_E \\ S_{23} & C_{23} & 0 & z_E \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

donde se utiliza la notación compacta $C_1 = \cos \theta_1$, $S_1 = \sin \theta_1$, $C_{23} = \cos(\theta_2 + \theta_3)$ y $S_{23} = \sin(\theta_2 + \theta_3)$.

El vector de posición del efector final $P_E = [x_E, y_E, z_E]^T$ está dado por:

$$x_E = \cos \theta_1 (a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)) + d_4 \sin \theta_1 \quad (6)$$

$$y_E = \sin \theta_1 (a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)) - d_4 \cos \theta_1 \quad (7)$$

$$z_E = d_1 + d_2 + a_3 \sin \theta_2 + a_4 \sin(\theta_2 + \theta_3) \quad (8)$$

Para verificar la consistencia del modelo matemático desarrollado, se evalúan las ecuaciones de cinemática directa bajo la configuración de reposo del prototipo (con todas las articulaciones en sus posiciones centrales $\theta_1 = 0$, $\theta_2 = 0$,

$\theta_3 = 0$). En este estado, al sustituir los valores y las constantes físicas en (6)-(8), se obtiene:

$$x_E = a_2 + a_3 \cos(0) + a_4 \cos(0) = 14,915 + 146,190 + 160,823 = 321,928 \text{ mm} \quad (9)$$

$$y_E = -d_4 = 4,000 \text{ mm} \quad (10)$$

$$z_E = d_1 + d_2 + a_3 \sin(0) + a_4 \sin(0) = 60,000 + 36,173 + 0 + 0 = 96,173 \text{ mm} \quad (11)$$

Estos resultados analíticos coinciden exactamente con las coordenadas de la brida (efector final) obtenidas directamente del árbol de transformadas (TF) de la simulación virtual de ROS 2 ($X = 321,928 \text{ mm}$, $Y = 4,000 \text{ mm}$, $Z = 96,173 \text{ mm}$). Esto valida la precisión del modelo cinemático de 4 sistemas propuesto para el guiado de trayectorias.

C. Cinemática Inversa y Análisis de Múltiples Soluciones

La resolución de la cinemática inversa consiste en determinar los ángulos físicos de los servomotores ($\theta_1, \theta_2, \theta_3$) a partir de una posición cartesiana deseada $P_E = [x_E, y_E, z_E]^T$. Para ello, desacoplamos el efecto de la rotación de la base y del desfase transversal d_4 :

1) *Cálculo de θ_1 (Rotación de la Base)*: Proyectando el efector sobre el plano horizontal y compensando el desfase transversal d_4 , definimos el radio proyectado plano R_p :

$$R_p = \sqrt{x_E^2 + y_E^2 - d_4^2} \quad (12)$$

A partir del cual se calcula el ángulo de la base mediante:

$$\theta_1 = \text{atan2}(y_E, x_E) - \text{atan2}(-d_4, R_p) \quad (13)$$

2) *Cálculo de θ_3 (Elevación del Codo)*: Se define la proyección en el plano del brazo. El radio proyectado corregido r y la altura corregida respecto al hombro z' se formulan como:

$$r = R_p - a_2 \quad (14)$$

$$z' = z_E - (d_1 + d_2) \quad (15)$$

Aplicando la ley de los cosenos en el triángulo del plano del brazo para resolver el ángulo θ_3 :

$$r^2 + z'^2 = a_3^2 + a_4^2 + 2a_3a_4 \cos \theta_3 \quad (16)$$

Despejando el término del coseno se obtiene:

$$\cos \theta_3 = \frac{r^2 + z'^2 - a_3^2 - a_4^2}{2a_3a_4} = \Psi \quad (17)$$

Para la existencia de una solución física real, se debe cumplir $-1 \leq \Psi \leq 1$. La ecuación admite dos soluciones:

$$\theta_3 = \text{atan2}(\pm \sqrt{1 - \Psi^2}, \Psi) \quad (18)$$

donde el signo positivo (+) representa la configuración de **codo abajo** y el signo negativo (−) representa la de **codo arriba** (flexión natural hacia abajo en el modelo calibrado). Seleccionamos la configuración de **codo arriba** para evitar colisiones mecánicas del brazo con la mesa de trabajo.

3) *Cálculo de θ_2 (Elevación del Hombro)*: Una vez obtenido θ_3 , se resuelve para θ_2 definiendo las constantes auxiliares $k_1 = a_3 + a_4 \cos \theta_3$ y $k_2 = a_4 \sin \theta_3$:

$$\theta_2 = \text{atan2}(k_1 z' - k_2 r, k_1 r + k_2 z') \quad (19)$$

D. Jacobiano y Análisis de Singularidades

El Jacobiano lineal del manipulador $J_v \in \mathbb{R}^{3 \times 3}$ relaciona las velocidades articulares $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3]^T$ con la velocidad lineal del efector final. Definimos el radio horizontal proyectado $R_p = a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)$. Derivando parcialmente las ecuaciones de la cinemática directa (6)-(8):

$$J_v = \begin{bmatrix} -S_1 R_p + d_4 C_1 & -C_1(a_3 S_2 + a_4 S_{23}) & -C_1 a_4 S_{23} \\ C_1 R_p + d_4 S_1 & -S_1(a_3 S_2 + a_4 S_{23}) & -S_1 a_4 S_{23} \\ 0 & a_3 C_2 + a_4 C_{23} & a_4 C_{23} \end{bmatrix} \quad (20)$$

donde S_i , C_i , S_{23} y C_{23} representan la notación simplificada de los senos y cosenos directos.

Calculando analíticamente el determinante de J_v (desprejando el término menor de desfase d_4):

$$\det(J_v) = -a_3 a_4 (a_2 + a_3 \cos \theta_2 + a_4 \cos(\theta_2 + \theta_3)) \sin \theta_3 \quad (21)$$

Se identifican las dos configuraciones singulares críticas:

- **Singularidad del codo** ($\sin \theta_3 = 0$): Ocurre cuando $\theta_3 = 0$ o $\theta_3 = -\pi$ (brazo completamente estirado o retraído).
- **Singularidad de alineación con el eje de la base** ($R_p = 0$): Ocurre cuando el efector final coincide con el eje de rotación de la base (Z_0).

IV. METODOLOGÍA Y ALCANCE

La ejecución del proyecto se estructuró en dos fases metodológicas consecutivas:

A. Fase 1: Modelado Mecánico y Simulación Virtual

Se diseñaron las piezas tridimensionales del manipulador en CAD. Los modelos tridimensionales de los eslabones se exportaron en formato STL para el proceso de manufactura, y se compilaron en un archivo descriptor URDF (con formato de mallas colisionables e inerciales). En el entorno de ROS 2 y Gazebo, se programó un nodo de simulación independiente para validar computacionalmente los modelos de cinemática directa e inversa, garantizando que el seguimiento de las trayectorias cartesianas de corte ocurra libre de colisiones mecánicas directas o singularidades. Esta simulación en ROS 2 y Gazebo se mantiene de forma puramente virtual, sin establecer conexiones directas de comunicación o control hacia el prototipo físico.

B. Fase 2: Fabricación e Integración de Hardware

La estructura mecánica se fabricó mediante impresión 3D FDM de los archivos STL utilizando plástico PETG con un patrón de relleno de alta densidad (40 % o superior) para absorber esfuerzos de flexión. Se integraron los actuadores en sus correspondientes articulaciones físicas: un servomotor MG996R en la base (giro izquierdo/derecha), un servomotor MG996R en el hombro (para soportar el torque gravitacional) y un servomotor MG90S en el codo.

V. IMPLEMENTACIÓN DE SOFTWARE Y ALGORITMOS DE CONTROL

El ecosistema de software del manipulador se divide en tres capas fundamentales que operan coordinadamente de forma puramente virtual para el entorno de simulación (la interfaz web interactiva de usuario, el generador síncrono de trayectorias en ROS 2 y el puente de comunicación de red):

A. Arquitectura de Comunicación e Intercambio de Datos

El ecosistema de software opera en un esquema coordinado donde la interfaz de usuario interactiva y el resolutor dinámico de la simulación física se comunican en tiempo real a través de un puente HTTP y tópicos de ROS 2. El flujo de datos e integración de los componentes clave se describe a continuación:

- **Capa de Interfaz Gráfica (Frontend):** Implementada en HTML/JS (`app.js`), captura la interacción del operador mediante barras de desplazamiento para los ángulos articulares o entradas cartesianas tridimensionales de posición (X, Y, Z) calculadas localmente mediante cinemática inversa. Cuando ocurre un cambio, la función `sendJointsToROS()` serializa los ángulos de las articulaciones J_1, J_2, J_3 (en grados sexagesimales) dentro de un objeto JSON y realiza una petición HTTP POST throttleada (con un intervalo mínimo de 50 ms) hacia el endpoint local de la API `/api/move`.
- **Capa del Puente de Red (Web Bridge):** El script en Python `web_bridge.py` inicializa de forma concurrente un servidor HTTP en el puerto 8080 y un nodo de ROS 2. Al recibir la petición POST en la ruta `/api/move`, el manejador de peticiones del servidor decodifica el objeto JSON, recupera los valores angulares y ejecuta la función `publish_joints()` del nodo. Esta función convierte de forma instantánea las variables articulares de grados sexagesimales a radianes y las publica en el tópico `/arm_controller/commands` de ROS 2.
- **Capa de Simulación Virtual (ROS 2 / Gazebo):** El resolutor físico y dinámico de Gazebo está suscrito de forma nativa a `/arm_controller/commands`. Al recibir los radianes de consigna, los controladores de posición articulares virtuales aplican torques proporcionales para llevar los eslabones simulados del robot a la pose deseada.
- **Generación de Trayectorias Autónomas:** De manera paralela e independiente de la interfaz gráfica, el script `trajectory_generator.py` publica directamente en el tópico `/arm_controller/commands` a una frecuencia de 50 Hz. Este nodo interpola entre una secuencia programada de keyframes utilizando un perfil de velocidad cosenoidal (S-curve), permitiendo realizar validaciones dinámicas automáticas dentro del simulador físico Gazebo de forma desacoplada del frontend web.

B. Interfaz Web de Control e Interacción Dinámica (`app.js`)

La interfaz de usuario está desarrollada en JavaScript de cliente puro y HTML5. Permite realizar simulación en línea y resolutor de cinemática inversa interactiva. El algoritmo detallado de la cinemática inversa se presenta en el Código 1 en el Apéndice A. Los componentes de la capa de interfaz web realizan las siguientes tareas:

- **Lienzo de Dibujo Interactivo (Canvas):** La rutina `drawRobot()` proyecta en tiempo real el comportamiento dinámico bidimensional del robot sobre un lienzo Canvas HTML5, mostrando vistas superior (X - Y) y lateral (X - Z) para dar retroalimentación visual al operador sobre la trayectoria de corte.
- **Interpolación de Suavizado de Movimiento:** Para transicionar sin saltos discretos hacia las coordenadas indicadas, la función `animateToJoints()` aplica un algoritmo de facilidad cúbica (*Cubic Ease-Out*) a 60 cuadros por segundo, emulando el amortiguamiento físico de los servomotores reales.

C. Generador de Trayectorias de Simulación (`trajectory_generator.py`)

El nodo generador de trayectorias en Python modela el perfil cinemático de interpolación de simulación en ROS 2 y Gazebo. El lazo periódico de control temporal de este script se detalla en el Código 2 en el Apéndice B. Este nodo realiza la generación y control síncrono mediante:

- **Interpolación Cosenoidal (S-curve):** La ecuación cosenoidal suaviza el incremento de velocidad al inicio y finalización del movimiento de cada eslabón, eliminando las aceleraciones abruptas que provocan oscilaciones inerciales o colisiones mecánicas virtuales en el resolutor de física de Gazebo.
- **Bucle Síncrono a 50 Hz:** Garantiza una comunicación continua hacia los controladores articulares virtuales a intervalos exactos de 20 ms.

D. Puente de Comunicación ROS 2 - Interfaz Web (`web_bridge.py`)

La comunicación intermedia entre la simulación y la interfaz de usuario web interactiva se gestiona mediante un puente de traducción bidireccional desarrollado en Python. La conversión de unidades y publicación de comandos se realiza a través del método detallado en el Código 3 en el Apéndice C. Este puente opera sobre las siguientes bases técnicas:

- **Servidor HTTP Multihilo:** Inicia un servidor web HTTP concurrente en el puerto 8080. La interfaz de usuario realiza peticiones de tipo POST con formato JSON hacia la ruta `'/api/move'`.
- **Traducción de Coordenadas Articulares:** Convierte las entradas angulares desde el formato en grados sexagesimales usado por la interfaz web al formato nativo en radianes esperado por los controladores virtuales de ROS 2, garantizando la sintonía espacial entre la simulación interactiva y el gemelo virtual.

E. Procesamiento de Imagen y Generación de Trayectoria 3D (*web_bridge.py*)

Para permitir que el brazo robótico realice trayectorias de corte basadas en dibujos o patrones de usuario, se implementó una canalización (pipeline) de visión artificial en el puente de comunicación:

- **Esqueletización de Imagen (Zhang-Suen):** Reduce los trazos de dibujo binarios de la imagen subida a líneas continuas de un solo píxel de espesor utilizando una implementación vectorizada en NumPy del algoritmo de adelgazamiento de Zhang-Suen, detallada en el Código 11 en el Apéndice I. Esto optimiza drásticamente el tiempo de procesamiento en comparación con bucles tradicionales.
- **Proyección a Coordenadas Cartesianas 3D:** Los píxeles esqueletizados en 2D se proyectan al plano tridimensional utilizando el modelo geométrico de la superficie de trabajo cilíndrica. Cada píxel se asocia a un punto tridimensional (x, y, z) en metros y almacena la instrucción de encendido del emisor láser.

F. Orquestación y Lanzamiento de Simulación (*gazebo.launch.py*)

La configuración y el inicio coordinado de los nodos de simulación física en ROS 2 se gestionan mediante un archivo de lanzamiento desarrollado en Python. El flujo secuencial y la configuración del entorno se detallan en el Código 4 en el Apéndice D. Este orquestador realiza las siguientes operaciones:

- **Carga y Procesamiento Dinámico de URDF/Xacro:** El script utiliza la utilidad `xacro.process_file()` para interpretar el modelo XML parametrizado. Posteriormente, aplica expresiones regulares para remover los comentarios XML redundantes con el fin de prevenir errores en el parser de ROS 2 y evitar inconsistencias en el plugin de Gazebo.
- **Gestión de Rutas de Modelos de Gazebo:** Actualiza dinámicamente la variable de entorno `GAZEBO_MODEL_PATH` para incluir el directorio local de mallas STL del brazo, garantizando la carga correcta de las geometrías de visualización en Gazebo Classic.
- **Secuenciación de Nodos mediante Manejo de Eventos:** Para evitar condiciones de carrera donde los controladores articulares intentan enlazarse antes de que el robot exista físicamente en la física del simulador, el script utiliza manejadores de eventos `RegisterEventHandler` y `OnProcessExit`. Esto fuerza que el controlador de posición de las articulaciones (`arm_controller`) se lance únicamente después de que el robot haya sido instanciado por el nodo `spawn_entity`.

G. Modelo Descriptor de la Estructura y Actuadores (*arm.urdf.xacro*)

El diseño estructural, los límites de movimiento y la configuración de hardware de control del gemelo virtual se definen

jerárquicamente en un archivo descriptor Xacro, presentado en el Código 5 en el Apéndice E. Este descriptor se estructura a partir de los siguientes elementos clave:

- **Geometría de Eslabones y Mallas STL:** Vincula directamente los archivos STL exportados de CAD (`base.stl`, `link1.stl`, etc.) escalados a metros (factor 0,001), definiendo la apariencia visual del robot en el simulador.
- **Límites y Dinámica de Articulaciones:** Modela los límites físicos angulares en radianes equivalentes a los límites de los servomotores físicos del prototipo. Adicionalmente, incluye parámetros dinámicos de fricción y amortiguamiento (`friction` y `damping`) para modelar el rozamiento de las articulaciones en Gazebo.
- **Integración con `ros2_control`:** Declara el elemento `<ros2_control>` que mapea las interfaces de comando y de estado (posiciones articulares y velocidades) del resolutor de física de Gazebo Classic (`gazebo_ros2_control/GazeboSystem`), permitiendo la inyección directa de señales desde el puente de comunicación HTTP. Esta declaración se complementa en la arquitectura de ROS 2 con el archivo de configuración `controllers.yaml`, el cual especifica la tasa de actualización de los bucles de control a 100 Hz y define un controlador síncrono del tipo `JointGroupPositionController` asociado a las tres articulaciones activas del robot.

H. Automatización de Configuración y Compilación (*setup.sh*)

Para simplificar la inicialización del entorno y garantizar la reproducibilidad de las pruebas, se programó un script de automatización en Bash. El funcionamiento y la lógica de este script se detallan en el Código 6 en el Apéndice F. Este script realiza las siguientes operaciones secuenciales:

- **Verificación del Entorno Aislado:** Comprueba la existencia del contenedor Distrobox (`ros2-humble`) y, en caso de no encontrarse activo, lo crea automáticamente instalando una base de Ubuntu 22.04.
- **Compilación Síncrona del Workspace:** Accede al contenedor, carga las variables de entorno de ROS 2 Humble y compila exclusivamente el paquete `arm_simulation` mediante el comando `colcon build`.
- **Generación de Scripts Lanzadores:** Escribe y otorga permisos de ejecución a los tres scripts rápidos (`launch_gazebo.sh`, `run_trajectory.sh` y `start_ui.sh`) en la raíz del proyecto para facilitar el uso diario.

I. Scripts Lanzadores del Entorno y Ejecutables

Para agilizar el flujo de trabajo del operador sin requerir la memorización de comandos de contenedor o de ROS 2, se desarrollaron tres scripts lanzadores. Su código fuente se presenta en el Código 7 en el Apéndice G. Cada script cumple una función específica dentro de la arquitectura distribuida:

- **Lanzamiento de Simulación (launch_gazebo.sh):** Limpia la variable de entorno `PATH` para remover rutas de Conda (previniendo conflictos de intérprete de Python con las dependencias de ROS 2), carga los entornos locales y ejecuta el launch de Gazebo.
- **Generador de Trayectorias (run_trajectory.sh):** Realiza la misma sanitización del entorno y ejecuta directamente el nodo controlador de interpolación cosenoidal.
- **Inicio de la Interfaz Gráfica (start_ui.sh):** Lanza de manera asíncrona el navegador predeterminado apuntando a `http://localhost:8080` y arranca concurrentemente el puente HTTP/ROS 2 (`web_bridge.py`) en segundo plano.

J. Firmware Embebido del Microcontrolador Físico (`main.cpp`)

Para el control directo y la coordinación de los actuadores del prototipo físico, se desarrolló un firmware embebido en C++ ejecutado sobre el microcontrolador ESP32. Su implementación completa se detalla en el Código 10 en el Apéndice H. Las principales funcionalidades de control y seguridad implementadas se describen a continuación:

- **Configuración de Periféricos e Inicialización de Actuadores:** El firmware utiliza la biblioteca `ESP32Servo` reservando tres temporizadores de hardware a una frecuencia de control de 50 Hz para generar las señales de modulación por ancho de pulso (PWM) destinadas a los servomotores de la base (GPIO 27), hombro (GPIO 14) y codo (GPIO 21). Adicionalmente, el pin GPIO 22 se define como una salida digital para la conmutación del emisor láser.
- **Resolvidor de Cinemática Inversa en Tiempo Real:** El microcontrolador ejecuta localmente el modelo matemático inverso analítico codo-arriba descrito en las ecuaciones (13)-(19). A partir de una coordenada cartesiana de destino (x, y, z) , calcula dinámicamente los ángulos en radianes $(\theta_1, \theta_2, \theta_3)$ y los traduce a microsegundos de ancho de pulso mediante la función `angleToMicroseconds()`, restringiendo los valores entre los límites físicos seguros de $[500, 2500] \mu s$.
- **Algoritmo de Movimiento Gradual Simultáneo:** Al comandar saltos instantáneos de posición, los servomotores demandan picos elevados de corriente instantánea que activan la protección contra sobrecorriente (OCP) de la fuente de alimentación externa, provocando el apagado del sistema. Para resolver este problema físico, se implementó el método `moveToTarget()`. Este algoritmo desplaza de forma síncrona y paralela los tres actuadores en incrementos máximos de $15 \mu s$ ($\approx 1,35^\circ$) espaciados por retardos temporales de 15 ms. Esto limita la tasa de variación de la corriente eléctrica durante los arranques y detenciones de las articulaciones.
- **Decodificación de Tramas y Auto-escalado de Unidades:** El bucle de control analiza las tramas provenientes de la interfaz serial esperando el formato de texto plano `"X:val,Y:val,Z:val"`. Con el fin de flexibilizar la

operación, el programa comprueba si las coordenadas ingresadas están expresadas en milímetros (valores absolutos mayores a 2,0). Si es el caso, aplica automáticamente un factor de escala de 0,001 para convertirlas a metros antes de realizar el cálculo cinemático.

- **Mecanismos de Seguridad Integrados:** Para prevenir quemaduras incidentales o incendios en la pieza de trabajo por pérdida de conexión con la interfaz de control, el código implementa un temporizador de watchdog por software. Si transcurren más de 1000 ms sin recibir comandos seriales válidos, el emisor láser se desactiva inmediatamente (`digitalWrite(LASER_PIN, LOW)`). Asimismo, la rutina `triggerEmergency()` permite interrumpir de forma inmediata la señal PWM y apagar el láser ante la recepción de tramas manuales de emergencia como `ESTOP` o `EMERGENCY`.

K. Scripts de Validación Cinemática y Alineación de Marcos

Con el fin de verificar la corrección del URDF, el árbol de transformaciones (TF) y las ecuaciones cinemáticas antes de operar el hardware, se programaron diversos scripts de prueba en Python:

- **Prueba Cinemática y Comparación DH (test_kinematics.py):** Compara analíticamente dos formulaciones de Denavit-Hartenberg (una estándar y otra con desfases angulares compensados ϕ_2 y ϕ_3), validando la consistencia matemática de la cinemática directa. El código se detalla en el Código 12 en el Apéndice J.
- **Validación de Alineación TF en Gazebo (test_tf_alignment.py):** Se conecta como nodo de ROS 2 para leer las transformaciones del árbol de TFs en tiempo real de la simulación activa. Comprueba que en la pose cero de calibración del robot, todos los ejes longitudinales X de las articulaciones sean coplanares y paralelos (Ángulo Pitch $\approx 0^\circ$ y desviación lateral $Y \approx 0$). Su código se presenta en el Código 13 en el Apéndice K.
- **Verificación Analítica de URDF (test_urdf_tf_alignment.py):** Realiza un análisis algebraico de las transformaciones de rotación RPY declaradas en el modelo descriptor URDF/Xacro, comprobando si los ejes longitudinales X resultan paralelos y horizontales en el plano de movimiento vertical. Su código se detalla en el Código 14 en el Apéndice L.
- **Lectura de Marcos de Referencia Activos (get_tf_distances.py):** Consulta el servicio de transformaciones de ROS 2 en tiempo real para imprimir las traslaciones y cuaterniones de rotación de cada eslabón con respecto a la base del robot. Su implementación se muestra en el Código 15 en el Apéndice M.
- **Parser de Geometría Xacro (get_gazebo_distances.py):** Analiza el árbol XML del archivo Xacro descriptor del brazo para extraer

automáticamente los límites físicos, desfases de juntas y longitudes calculadas de los eslabones móviles. Su código se detalla en el Código 16 en el Apéndice N.

VI. SELECCIÓN DE ACTUADORES Y SENSORES

Para garantizar el éxito de la implementación física y su congruencia con el entorno de simulación, se detalla la selección de componentes de hardware:

A. Actuadores

La arquitectura cinemática de 3 GDL resulta idónea para este caso de estudio; dado que la herramienta es un haz láser unidireccional colineal con el eje de aproximación, y el objetivo es el corte y grabado sobre superficies cilíndricas, el control del posicionamiento cartesiano en los tres ejes (X, Y, Z) es crítico para compensar la curvatura de la pieza y mantener la distancia focal constante del haz. La omisión del control de orientación redundante (pitch/roll) disminuye drásticamente el torque estático requerido en las articulaciones proximales (base y hombro), lo que mitiga las deflexiones estructurales y maximiza la rigidez y repetibilidad del sistema. El diseño se complementa ubicando estratégicamente actuadores de baja masa en los eslabones distales para minimizar el tensor de inercia del mecanismo.

- **Actuador de rotación principal (Base):** Se implementó un servomotor de alto torque y engranajes metálicos (MG996R) en la base para accionar el giro del brazo a la izquierda y derecha. Este actuador soporta las cargas dinámicas y el momento de inercia de toda la estructura móvil.
- **Actuadores de elevación (Hombro y Codo):** Se utilizó un servomotor de alto torque MG996R para el hombro (articulación pegada a la base que hace elevar el brazo) para soportar el peso y los esfuerzos de flexión de los eslabones, y un servomotor de engranajes metálicos MG90S en la articulación del codo para controlar la elevación distal.
- **Efector final (Módulo Láser de estado sólido):** Consiste en un diodo láser de baja potencia con lógica transistor-transistor (TTL) de 5V, el cual es conmutado como un nivel lógico de entrada/salida de propósito general (GPIO) o vía PWM desde el ESP32. Esto permite una activación síncrona y de baja latencia acoplada al perfil cinemático de la trayectoria, emulando la dinámica de interpolación de un proceso industrial de corte láser CNC.

B. Sensores

- **Potenciómetros internos:** Los servomotores ya incluyen sensores de posición (potenciómetros) internamente. Esto mantiene el diseño mecánico limpio ya que no se requiere acoplar encoders ópticos externos para leer los ángulos de las articulaciones, eliminando la necesidad de cableado para sensores externos de posición o corriente.

C. Especificación de Componentes

En la Tabla II se detallan las especificaciones y funciones de los componentes de hardware seleccionados.

Tabla II
ESPECIFICACIONES DE LOS COMPONENTES DE HARDWARE

Componente	Modelo	Función en el Sistema
Controlador	ESP32	Control de bajo nivel y PWM.
Actuador base	MG996R (1 ud.)	Rotación izquierda/derecha.
Actuador hombro	MG996R (1 ud.)	Elevación del hombro.
Actuador codo	MG90S (1 ud.)	Elevación del codo.
Efector final	Diodo Láser (5V)	Haz de corte de baja potencia.
Estructura	PETG	Piezas impresas en 3D.
Alimentación	Fuente 5V 5A	Energía para actuadores y láser.

D. Presupuesto de Masa y Dimensiones

En la Tabla III se detalla la masa y dimensiones físicas estimadas para los diferentes componentes del brazo manipulador.

Tabla III
PRESUPUESTO DE MASA Y DIMENSIONES DE LOS COMPONENTES

Componente	Masa (g)	Cant.	Dimensiones (mm)
ESP32 NodeMCU	10	1	48.0 × 26.0 × 12.0
Servo MG996R	55	2	40.7 × 19.7 × 42.9
Servo MG90S	13.4	1	22.8 × 12.2 × 28.5
Diodo Láser 5V	10	1	φ 12.0 × 35.0
Estructura 3D (PETG)	350	1	Envolvente ≈ 250 × 150
Fuente 5V 5A	150	1	110.0 × 78.0 × 36.0
Cables y tornillería	50	1	N/A
Masa Total	693 g	(≈ 0.69 kg)	

E. Presupuesto y Consumo de Energía

La Tabla IV detalla el voltaje nominal y las corrientes de funcionamiento promedio y de pico para estimar la potencia del sistema.

Tabla IV
PRESUPUESTO Y CONSUMO DE ENERGÍA DE LOS COMPONENTES

Componente	Voltaje (V)	Corr. Prom.	Corr. Pico
ESP32	5.0	120 mA	240 mA
MG996R (2 ud.)	5.0	1000 mA	5.0 A
MG90S (1 ud.)	5.0	180 mA	0.8 A
Láser 5V	5.0	30 mA	30 mA
Consumo Promedio		1.33 A (≈ 6.7 W)	
Consumo Pico		6.07 A (≈ 30.4 W)	

F. Estimación de Costos

El presupuesto estimado para la adquisición de los componentes de hardware requeridos se presenta en la Tabla V.

Tabla V
PRESUPUESTO Y ESTIMACIÓN DE COSTOS

Descripción	Cant.	Unit. (\$/.)	Total (\$/.)
ESP32 NodeMCU	1	25.00	25.00
Servomotor MG996R	2	30.00	60.00
Servomotor MG90S	1	12.00	12.00
Módulo Láser 5V TTL	1	15.00	15.00
Filamento PETG (1 kg)	1	70.00	70.00
Fuente de alimentación 5V 5A	1	35.00	35.00
Cables y tornillería	-	15.00	15.00
Total Estimado			\$/ 232.00

G. Esquema de Conexiones Eléctricas

La integración física del sistema requiere una arquitectura de alimentación y control claramente definida. El ESP32 es alimentado directamente por la laptop vía USB, mientras que los servomotores y el diodo láser reciben su tensión de operación de +5 V desde la fuente de PC. El encendido del láser se gestiona mediante un transistor NPN 2N2222 con resistencia de base de 220 Ω , accionado por el pin GPIO 25 del ESP32. La Fig. 2 muestra el esquema completo de conexiones.

VII. APLICACIÓN INDUSTRIAL

A diferencia de los pantógrafos y cortadoras láser cartesianas convencionales, los cuales están limitados a trazar trayectorias coplanares bidimensionales (2D) en láminas planas, la configuración antropomórfica de 3 GDL permite posicionar la herramienta en un espacio tridimensional (X, Y, Z). Esta característica habilita la aplicación del robot en procesos industriales de corte y grabado sobre superficies irregulares

Esquema de Conexiones Eléctricas
Prototipo del Brazo Robótico Láser

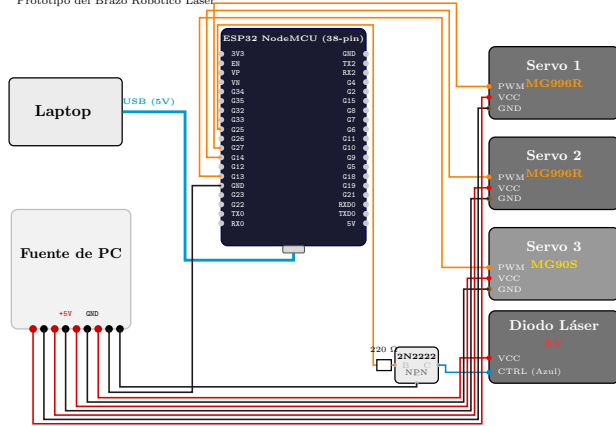


Figura 2. Esquema de conexiones eléctricas del prototipo. Se detallan las líneas de alimentación desde la fuente de PC (+5V y GND) hacia los servomotores Servo 1 (MG996R), Servo 2 (MG996R) y Servo 3 (MG90S), el circuito de control del diodo láser con transistor NPN 2N2222 y resistencia de base de 220 Ω , y la conexión USB de alimentación del ESP32 NodeMCU de 38 pines.

y no coplanares, tales como el perfilado de cuerpos curvados y el calado de piezas cilíndricas.

En la industria de manufactura mecánica, el procesamiento de piezas no planas requiere realizar cortes y grabados con perfiles tridimensionales complejos (por ejemplo, intersecciones de geometrías cilíndricas o marcas en piezas curvas). Al colocar un haz láser en el efector final del robot, es posible realizar este proceso sin contacto mecánico directo. A partir del modelo de la superficie cilíndrica de una pieza de radio R_{tubo} colocada de forma horizontal a lo largo del eje Y_0 a una distancia x_{tubo} y a una altura de desfase z_{tubo} , la coordenada vertical de corte z_E se calcula dinámicamente en función de la posición radial x_E :

$$z_E = z_{tubo} + \sqrt{R_{tubo}^2 - (x_E - x_{tubo})^2} \quad (22)$$

El resolutor de cinemática inversa desarrollado en (13)-(19) calcula de manera instantánea las posiciones articulares ($\theta_1, \theta_2, \theta_3$) para que el láser siga esta trayectoria no planar, manteniendo la distancia focal constante respecto a la pared de la pieza. Esto demuestra el valor industrial del manipulador propuesto frente a las soluciones cartesianas convencionales de bajo costo, ampliando el alcance de automatización hacia geometrías complejas no convencionales.

APÉNDICE A

CÓDIGO DEL RESOLVEDOR DE CINEMÁTICA INVERSA EN LA INTERFAZ WEB (APP.JS)

```

1 function solveIK(x, y, z) {
2   const targetX = parseFloat(x);
3   const targetY = parseFloat(y);
4   const targetZ = parseFloat(z);
5
6   if (isNaN(targetX) || isNaN(targetY) || isNaN(targetZ)) {
7     return { error: 'VALORES_INVALIDOS', msg: 'Ingresa valores numericos validos.' };
8   }
9
10  let r = Math.sqrt(targetX * targetX + targetY * targetY);
11  if (r < Math.abs(d4)) {
12    r = Math.abs(d4);
13  }
14  const Rp = Math.sqrt(r * r - d4 * d4);
15  const thetal = (targetX === 0 && targetY === 0) ? -Math.PI / 2 : Math.atan2(
16    targetY, targetX) - Math.atan2(-d4, Rp);
17  const j1 = thetal * 180 / Math.PI;
18
19  if (j1 < jointsConfig[1].min || j1 > jointsConfig[1].max) {
20    return { error: 'FUERA DE ALCANCE', msg: 'Rotacion base J1 fuera de limites (-165 a 165).'; };
21  }
22
23  const xc = Rp - a2;
24  const zc = targetZ - (d1 + d2);
25
26  const psi_numerator = xc * xc + zc * zc - L1 * L1 - L2 * L2;
27  const psi_denominator = 2.0 * L1 * L2;
28  let psi = psi_numerator / psi_denominator;
29
30  if (psi > 1.0 && psi <= 1.005) {
31    psi = 1.0;
32  } else if (psi < -1.0 && psi >= -1.005) {
33    psi = -1.0;
34  }
35
36  if (psi < -1.0 || psi > 1.0) {
37    return { error: 'FUERA DE ALCANCE', msg: 'Coordenadas fuera del alcance del brazo.' };
38  }
39
40  const solutions = [-1, 1];
41  for (let s of solutions) {
42    const sinTheta3_star = s * Math.sqrt(1.0 - psi * psi);
43    const theta3_star = Math.atan2(sinTheta3_star, psi);
44
45    const k1 = L1 + L2 * Math.cos(theta3_star);
46    const k2 = L2 * Math.sin(theta3_star);
47    const theta2_star = Math.atan2(k1 * zc - k2 * xc, k1 * xc + k2 * zc);
48
49    const theta2 = theta2_star;
50    const theta3 = theta3_star;
51
52    const j2 = theta2 * 180 / Math.PI;
53    const j3 = theta3 * 180 / Math.PI;

```



```

53
54     if (j2 >= jointsConfig[2].min && j2 <= jointsConfig[2].max &&
55         j3 >= jointsConfig[3].min && j3 <= jointsConfig[3].max) {
56         return { j1, j2, j3 };
57     }
58 }
59
60 return { error: 'FUERA DE ALCANCE', msg: 'Posicion cinematicamente inalcanzable
61         por límites de articulacion.' };

```

Listing 1. Resolvedor interactivo de cinemática inversa en JS (app.js).

APÉNDICE B

CÓDIGO DEL GENERADOR DE TRAYECTORIAS DE SIMULACIÓN (TRAJECTORY_GENERATOR.PY)

```

1  #!/usr/bin/env python3
2  import rclpy
3  from rclpy.node import Node
4  from std_msgs.msg import Float64MultiArray
5  import time
6  import math
7
8  class TrajectoryGenerator(Node):
9      def __init__(self):
10         """
11         Inicializa el nodo de ROS 2, configura el publicador de comandos de control
12         , define los keyframes de la secuencia de corte industrial y arranca el
13         temporizador de interpolacion.
14         """
15         super().__init__('trajectory_generator')
16         self.publisher_ = self.create_publisher(Float64MultiArray, '/arm_controller
17         /commands', 10)
18
19         self.rate = 50.0
20         self.timer = self.create_timer(1.0 / self.rate, self.timer_callback)
21
22         self.keyframes = [
23             {"pose": [0.0, 0.0, 0.0], "time": 3.0},
24             {"pose": [-0.6, 0.4, -0.4], "time": 3.0},
25             {"pose": [0.6, 0.4, -0.4], "time": 4.0},
26             {"pose": [0.6, 0.6, -0.7], "time": 2.0},
27             {"pose": [-0.6, 0.6, -0.7], "time": 4.0},
28             {"pose": [0.0, 0.0, 0.0], "time": 3.0}
29         ]
30
31         self.current_keyframe_idx = 0
32         self.step_counter = 0
33         self.total_steps_for_transition = 0
34         self.start_pose = [0.0, 0.0, 0.0]
35         self.target_pose = self.keyframes[0]["pose"]
36
37         self.get_logger().info("Nodo de trayectoria inicializado. Iniciando ciclo
38         de corte...")
39         self.prepare_next_transition()
40
41         def prepare_next_transition(self):
42             """
43             Prepara los parametros cinematicos y de interpolacion para la transicion
44             hacia el
45             siguiente keyframe de la secuencia, incrementando el indice de trayectoria.
46             """
47             self.start_pose = list(self.target_pose)
48             keyframe = self.keyframes[self.current_keyframe_idx]
49             self.target_pose = keyframe["pose"]
50             duration = keyframe["time"]
51
52             self.total_steps_for_transition = int(duration * self.rate)
53             self.step_counter = 0
54
55             self.get_logger().info(
56                 f"Transicionando a Keyframe {self.current_keyframe_idx}: {self.
57                 target_pose} en {duration}s"
58             )
59
60             self.current_keyframe_idx = (self.current_keyframe_idx + 1) % len(self.
61             keyframes)
62
63         def timer_callback(self):
64             """
65             Ejecuta periodicamente la interpolacion cosenoidal (S-curve) entre poses
66             conjuntas
67             sucesivas y publica el vector de comandos correspondiente a la tasa del
68             bucle.
69             """
70             if self.step_counter >= self.total_steps_for_transition:
71                 self.prepare_next_transition()
72
73             t = float(self.step_counter) / float(self.total_steps_for_transition)
74             smooth_t = (1.0 - math.cos(t * math.pi)) / 2.0
75
76             current_command = []
77             for start, target in zip(self.start_pose, self.target_pose):
78                 val = start + (target - start) * smooth_t
79                 current_command.append(val)
80
81             msg = Float64MultiArray()
82             msg.data = current_command
83             self.publisher_.publish(msg)
84
85             self.step_counter += 1
86
87     def main(args=None):

```

```

81     """
82     Funcion de entrada que inicializa el entorno de ROS 2, instancia el generador
83     de trayectoria,
84     mantiene el ciclo de ejecucion activo y maneja el apagado limpio del nodo.
85     """
86     rclpy.init(args=args)
87     node = TrajectoryGenerator()
88     try:
89         rclpy.spin(node)
90     except KeyboardInterrupt:
91         node.get_logger().info("Generador de trayectoria detenido por el usuario.")
92     finally:
93         node.destroy_node()
94         rclpy.shutdown()
95
96     if __name__ == '__main__':
97         main()

```

Listing 2. Generación de perfiles S-curve por interpolación cosenoidal y control de keyframes (trajectory_generator.py).

APÉNDICE C

CÓDIGO DEL PUENTE DE COMUNICACIÓN ROS 2 - INTERFAZ WEB (WEB_BRIDGE.PY)

```

1  def publish_joints(self, j1_deg, j2_deg, j3_deg):
2      j1_rad = float(j1_deg) * math.pi / 180.0
3      j2_rad = float(j2_deg) * math.pi / 180.0
4      j3_rad = float(j3_deg) * math.pi / 180.0
5      msg = Float64MultiArray()
6      msg.data = [j1_rad, j2_rad, j3_rad]
7      self.publisher_.publish(msg)
8      self.get_logger().info(f'Publicado en ROS -> J1: {j1_rad:.4f} rad, J2: {j2_rad
9      :.4f} rad, J3: {j3_rad:.4f} rad')

```

Listing 3. Conversión de unidades y envío al simulador (web_bridge.py).

APÉNDICE D

CÓDIGO DEL LANZAMIENTO DE SIMULACIÓN EN ROS 2 (GAZEBO.LAUNCH.PY)

```

1  import os
2  from ament_index_python.packages import get_package_share_directory
3  from launch import LaunchDescription
4  from launch.actions import IncludeLaunchDescription, RegisterEventHandler
5  from launch.event_handlers import OnProcessExit
6  from launch.launch_description_sources import PythonLaunchDescriptionSource
7  from launch_ros.actions import Node
8  import xacro
9
10 def generate_launch_description():
11     pkg_share = get_package_share_directory('arm_simulation')
12     xacro_file = os.path.join(pkg_share, 'urdf', 'arm.urdf.xacro')
13
14     pkg_share_parent = os.path.dirname(pkg_share)
15     default_gazebo_models = '/usr/share/gazebo-11/models:/usr/share/gazebo/models'
16     if 'GAZEBO_MODEL_PATH' in os.environ:
17         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
18         default_gazebo_models + ':' + os.environ['GAZEBO_MODEL_PATH']
19     else:
20         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
21         default_gazebo_models
22
23     os.environ['GAZEBO_MODEL_DATABASE_URI'] = ''
24
25     import re
26     robot_description_config = xacro.process_file(xacro_file)
27     robot_desc = robot_description_config.toxml()
28     robot_desc = re.sub(r'<!--.*?-->', '', robot_desc, flags=re.DOTALL)
29
30     robot_state_publisher_node = Node(
31         package='robot_state_publisher',
32         executable='robot_state_publisher',
33         name='robot_state_publisher',
34         output='screen',
35         parameters=[{'robot_description': robot_desc, 'use_sim_time': True}])
36
37     gazebo = IncludeLaunchDescription(
38         PythonLaunchDescriptionSource([os.path.join(
39             get_package_share_directory('gazebo_ros'), 'launch', 'gazebo.launch.py'
40         )]),
41         launch_arguments={'verbose': 'true'}.items()
42     )
43
44     spawn_entity = Node(
45         package='gazebo_ros',
46         executable='spawn_entity.py',
47         arguments=['-topic', 'robot_description', '-entity', 'arm_3gdl', '-timeout',
48         '120'],
49         output='screen'
50     )
51
52     spawn_joint_state_broadcaster = Node(
53         package="controller_manager",
54         executable="spawner",
55         arguments=["joint_state_broadcaster", "--controller-manager", "/
56         controller_manager"],
57         output="screen"
58     )

```

```

55
56 spawn_arm_controller = Node(
57     package="controller_manager",
58     executable="spawner",
59     arguments=["arm_controller", "--controller-manager", "/controller_manager"
60 ],
61     output='screen'
62 )
63
64 return LaunchDescription([
65     gazebo,
66     robot_state_publisher_node,
67     spawn_entity,
68     RegisterEventHandler(
69         event_handler=OnProcessExit(
70             target_action=spawn_entity,
71             on_exit=[spawn_joint_state_broadcaster],
72         )
73     ),
74     RegisterEventHandler(
75         event_handler=OnProcessExit(
76             target_action=spawn_joint_state_broadcaster,
77             on_exit=[spawn_arm_controller],
78         )
79     )
80 ])

```

Listing 4. Script de lanzamiento y control secuencial en ROS 2 (gazebo.launch.py).

APÉNDICE E

ARCHIVO DE DESCRIPCIÓN ESTRUCTURAL Y ACTUADORES XACRO (ARM.URDF.XACRO)

```

1 <?xml version="1.0"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm_3gd1">
3
4 <!-- ===== Materiales de Visualizacion ===== -->
5 <material name="abb_orange">
6   <color rgba="1.0 0.45 0.0 1.0"/>
7 </material>
8 <material name="abb_black">
9   <color rgba="0.12 0.12 0.12 1.0"/>
10 </material>
11 <material name="silver">
12   <color rgba="0.75 0.75 0.75 1.0"/>
13 </material>
14 <material name="graphite">
15   <color rgba="0.25 0.25 0.25 1.0"/>
16 </material>
17 <material name="brass">
18   <color rgba="0.8 0.6 0.2 1.0"/>
19 </material>
20 <material name="laser_red">
21   <color rgba="1.0 0.0 0.0 0.8"/>
22 </material>
23
24 <!-- ===== Base Fija (Mundo) ===== -->
25 <link name="world"/>
26 <joint name="world_to_base" type="fixed">
27   <parent link="world"/>
28   <child link="base_link"/>
29   <origin xyz="0 0 0" rpy="1.57079632679 0 0"/>
30 </joint>
31
32 <!-- ===== Eslabon Base ===== -->
33 <link name="base_link">
34   <inertial>
35     <mass value="1.0"/>
36     <origin xyz="0 0 0" rpy="0 0 0"/>
37     <inertia ixx="0.005" ixy="0.0" izx="0.0" iyy="0.005" iyz="0.0" izz="0.005"/>
38   </inertial>
39   <visual>
40     <origin xyz="-0.691316 -0.869483 -1.350860" rpy="0 0 0"/>
41     <geometry>
42       <mesh filename="package://arm_simulation/meshes/custom_arm/base.stl" scale=
43         "0.001 0.001 0.001"/>
44     </geometry>
45     <material name="abb_orange"/>
46   </visual>
47 </link>
48
49 <!-- ===== Articulacion 1 (Rotacion de Base) ===== -->
50 <joint name="joint1" type="revolute">
51   <origin rpy="-1.57079632679 0 0" xyz="0 0 0.060 0"/>
52   <parent link="base_link"/>
53   <child link="link1"/>
54   <axis xyz="0 0 1"/>
55   <limit lower="-2.87979" upper="2.87979" effort="20.0" velocity="1.0"/>
56   <dynamics damping="1.0" friction="1.0"/>
57 </joint>
58
59 <!-- ===== Eslabon 1 ===== -->
60 <link name="link1">
61   <inertial>
62     <mass value="0.5"/>
63     <origin xyz="0 0 0" rpy="0 0 0"/>
64     <inertia ixx="0.002" ixy="0.0" izx="0.0" iyy="0.002" iyz="0.0" izz="0.002"/>
65   </inertial>
66   <visual>
67     <origin xyz="-0.691316 1.350860 -0.929483" rpy="1.57079632679 0 0"/>
68     <geometry>
69       <mesh filename="package://arm_simulation/meshes/custom_arm/link1.stl" scale

```

```

70     <material name="abb_orange"/>
71   </visual>
72 </link>
73
74 <!-- ===== Articulacion 2 (Hombro) ===== -->
75 <joint name="joint2" type="revolute">
76   <origin rpy="1.57079632679 0 0" xyz="0.014915 0 0.036173"/>
77   <parent link="link1"/>
78   <child link="link2"/>
79   <axis xyz="0 0 1"/>
80   <limit lower="-1.91986" upper="1.91986" effort="25.0" velocity="1.0"/>
81   <dynamics damping="1.0" friction="1.0"/>
82 </joint>
83
84 <!-- ===== Eslabon 2 ===== -->
85 <link name="link2">
86   <inertial>
87     <mass value="0.5"/>
88     <origin xyz="0 0 0" rpy="0 0 0"/>
89     <inertia ixx="0.002" ixy="0.0" izx="0.0" iyy="0.002" iyz="0.0" izz="0.002"/>
90   </inertial>
91   <visual>
92     <origin xyz="-0.706231 -0.965655 -1.350860" rpy="0 0 0"/>
93     <geometry>
94       <mesh filename="package://arm_simulation/meshes/custom_arm/link2.stl" scale
95         ="0.001 0.001 0.001"/>
96     </geometry>
97     <material name="abb_orange"/>
98   </visual>
99 </link>
100
101 <!-- ===== Articulacion 3 (Codo) ===== -->
102 <joint name="joint3" type="revolute">
103   <origin rpy="0 0 0" xyz="0.075256 0.125332 0"/>
104   <parent link="link2"/>
105   <child link="link3"/>
106   <axis xyz="0 0 1"/>
107   <limit lower="-1.91986" upper="1.22173" effort="20.0" velocity="1.5"/>
108   <dynamics damping="1.0" friction="1.0"/>
109 </joint>
110
111 <!-- ===== Eslabon 3 ===== -->
112 <link name="link3">
113   <inertial>
114     <mass value="0.3"/>
115     <origin xyz="0 0 0" rpy="0 0 0"/>
116     <inertia ixx="0.001" ixy="0.0" izx="0.0" iyy="0.001" iyz="0.0" izz="0.001"/>
117   </inertial>
118   <visual>
119     <origin xyz="-0.781487 -1.090987 -1.350860" rpy="0 0 0"/>
120     <mesh filename="package://arm_simulation/meshes/custom_arm/link3.stl" scale
121       ="0.001 0.001 0.001"/>
122     </geometry>
123     <material name="abb_orange"/>
124   </visual>
125 </link>
126
127 <!-- ===== Acoplamiento de Herramienta (Flange y Tool0) ===== -->
128 <link name="flange"/>
129 <joint type="fixed" name="joint_3-flange">
130   <origin xyz="0.158 -0.030 -0.004" rpy="0 0 -0.191392"/>
131   <parent link="link3"/>
132   <child link="flange"/>
133 </joint>
134
135 <link name="tool0"/>
136 <joint name="flange-tool0" type="fixed">
137   <origin xyz="0 0 0" rpy="0 1.57079632679 0" />
138   <parent link="flange" />
139   <child link="tool0" />
140 </joint>
141
142 <!-- ===== Controladores y Plugins de Gazebo ===== -->
143 <gazebo>
144   <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
145     <parameters>$(find arm_simulation)/config/controllers.yaml</parameters>
146   </plugin>
147 </gazebo>
148
149 <ros2_control name="GazeboSystem" type="system">
150   <hardware>
151     <plugin>gazebo_ros2_control/GazeboSystem</plugin>
152   </hardware>
153   <joint name="joint1">
154     <command_interface name="position">
155       <param name="min">-2.87979</param>
156       <param name="max">2.87979</param>
157     </command_interface>
158     <state_interface name="position">
159       <param name="initial_value">0.0</param>
160     </state_interface>
161     <state_interface name="velocity"/>
162   </joint>
163   <joint name="joint2">
164     <command_interface name="position">
165       <param name="min">-1.91986</param>
166       <param name="max">1.91986</param>
167     </command_interface>
168     <state_interface name="position">
169       <param name="initial_value">0.0</param>
170     </state_interface>
171     <state_interface name="velocity"/>
172   </joint>
173   <joint name="joint3">
174     <command_interface name="position">
175       <param name="min">-1.91986</param>
176       <param name="max">1.22173</param>
177     </command_interface>

```

```

177     <state_interface name="position">
178       <param name="initial_value">0.0</param>
179     </state_interface>
180     <state_interface name="velocity"/>
181   </joint>
182 </ros2_control>
183
184 <!-- ===== Macro para Sistemas de Referencia (RGB) ===== -->
185 <xacro:macro name="visualize_frame" params="parent prefix xyz:= '0 0 0' rpy:= '0 0 0'
186   0' scale:=0.05">
187   <!-- Eje X (Rojo) -->
188   <link name="${prefix}_frame_x">
189     <inertial>
190       <mass value="0.0001"/>
191       <origin xyz="0 0 0" rpy="0 0 0"/>
192       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" izy="0" izz="1e-9"/>
193     </inertial>
194     <visual>
195       <origin xyz="${scale/4} 0 0" rpy="0 1.57079632679 0"/>
196       <geometry>
197         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
198       </geometry>
199       <material name="red">
200         <color rgba="1.0 0.0 0.0 0.8"/>
201       </material>
202     </visual>
203   </link>
204   <joint name="${prefix}_frame_x_joint" type="fixed">
205     <origin xyz="${xyz}" rpy="${rpy}"/>
206     <parent link="${parent}"/>
207     <child link="${prefix}_frame_x"/>
208   </joint>
209   <gazebo reference="${prefix}_frame_x">
210     <material>Gazebo/Red</material>
211   </gazebo>
212
213   <!-- Eje Y (Verde) -->
214   <link name="${prefix}_frame_y">
215     <inertial>
216       <mass value="0.0001"/>
217       <origin xyz="0 0 0" rpy="0 0 0"/>
218       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" izy="0" izz="1e-9"/>
219     </inertial>
220     <visual>
221       <origin xyz="0 ${scale/4} 0" rpy="-1.57079632679 0 0"/>
222       <geometry>
223         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
224       </geometry>
225       <material name="green">
226         <color rgba="0.0 1.0 0.0 0.8"/>
227       </material>
228     </visual>
229   </link>
230   <joint name="${prefix}_frame_y_joint" type="fixed">
231     <origin xyz="${xyz}" rpy="${rpy}"/>
232     <parent link="${parent}"/>
233     <child link="${prefix}_frame_y"/>
234   </joint>
235   <gazebo reference="${prefix}_frame_y">
236     <material>Gazebo/Green</material>
237   </gazebo>
238
239   <!-- Eje Z (Azul) -->
240   <link name="${prefix}_frame_z">
241     <inertial>
242       <mass value="0.0001"/>
243       <origin xyz="0 0 0" rpy="0 0 0"/>
244       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" izy="0" izz="1e-9"/>
245     </inertial>
246     <visual>
247       <origin xyz="0 0 ${scale/4}" rpy="0 0 0"/>
248       <geometry>
249         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
250       </geometry>
251       <material name="blue">
252         <color rgba="0.0 0.0 1.0 0.8"/>
253       </material>
254     </visual>
255   </link>
256   <joint name="${prefix}_frame_z_joint" type="fixed">
257     <origin xyz="${xyz}" rpy="${rpy}"/>
258     <parent link="${parent}"/>
259     <child link="${prefix}_frame_z"/>
260   </joint>
261   <gazebo reference="${prefix}_frame_z">
262     <material>Gazebo/Blue</material>
263   </gazebo>
264 </xacro:macro>
265
266 <!-- Instanciacion de los Sistemas de Referencia en cada Articulacion (D-H
267   Secuencial) -->
268 <xacro:visualize_frame parent="base_link" prefix="system0" xyz="0 0 0" rpy="
269   -1.57079632679 0 0" scale="0.08"/>
270 <xacro:visualize_frame parent="link1" prefix="system1" xyz="0 0 0" rpy="0 0 0"
271   scale="0.06"/>
272 <xacro:visualize_frame parent="link2" prefix="system2" xyz="0 0 0" rpy="0 0
273   1.030041" scale="0.06"/>
274 <xacro:visualize_frame parent="link3" prefix="system3" xyz="0 0 0" rpy="0 0
275   -0.191392" scale="0.06"/>
276 <xacro:visualize_frame parent="flange" prefix="system4" xyz="0 0 0" rpy="0 0 0"
277   scale="0.06"/>
278
279 <!-- ===== Camara de Monitoreo (Gazebo) ===== -->
280 <link name="camera_link">
281   <inertial>
282     <mass value="0.1"/>
283     <origin xyz="0 0 0" rpy="0 0 0"/>
284     <inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" izy="0" izz="1e-4"/>
285   </inertial>
286   <visual>
287     <!-- Cuerpo de la camara -->
288     <origin xyz="0 0 0" rpy="0 0 0"/>
289     <geometry>
290       <box size="0.05 0.05 0.05"/>
291     </geometry>
292     <material name="graphite"/>
293   </visual>
294   <!-- Lente de la camara -->
295   <origin xyz="0.03 0 0" rpy="0 1.57079632679 0"/>
296   <geometry>
297     <cylinder radius="0.015" length="0.02"/>
298   </geometry>
299   <material name="abb_black"/>
300 </link>
301
302 <joint name="camera_joint" type="fixed">
303   <!-- Ubicada sobre el eje X positivo de system0, conectada a base_link para
304     evitar fallos de spawn en Gazebo -->
305   <origin xyz="0.11 0.025 0.0" rpy="-1.57079632679 0 0"/>
306   <parent link="base_link"/>
307   <child link="camera_link"/>
308 </joint>
309
310 <!-- Plugin de Gazebo para transmitir la imagen en ROS 2 -->
311 <gazebo reference="camera_link">
312   <sensor type="camera" name="work_camera">
313     <update_rate>15.0</update_rate>
314     <visualize>true</visualize>
315     <camera name="head">
316       <horizontal_fov>1.085595</horizontal_fov>
317       <image>
318         <width>640</width>
319         <height>480</height>
320         <format>R8G8B8</format>
321       </image>
322       <clip>
323         <near>0.05</near>
324         <far>8.0</far>
325       </clip>
326     </camera>
327     <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
328       <frame_name>camera_link</frame_name>
329     </plugin>
330   </sensor>
331 </gazebo>
332
333 <!-- Sistemas de referencia de la camara (Base y Lente) -->
334 <xacro:visualize_frame parent="base_link" prefix="camera_base" xyz="0.11 0.0 0.0"
335   rpy="-1.57079632679 0 0" scale="0.04"/>
336 <xacro:visualize_frame parent="base_link" prefix="camera_lens" xyz="0.15 0.025
337   0.0" rpy="-1.57079632679 0 0" scale="0.02"/>
338
339 </robot>

```

Listing 5. Modelado estructural, cinemático y configuración de ros2_control en Xacro (arm.urdf.xacro).

APÉNDICE F SCRIPT DE AUTOMATIZACION DE CONFIGURACION Y COMPILACION (SETUP.SH)

```

1  #!/bin/bash
2  set -e
3  GREEN='\033[0;32m'
4  BLUE='\033[0;34m'
5  YELLOW='\033[1;33m'
6  RED='\033[0;31m'
7  NC='\033[0m'
8
9  echo -e "${BLUE}===== ${NC}"
10 echo -e "${GREEN}          KiraOne - SETUP & COMPILATION SYSTEM    ${NC}"
11 echo -e "${BLUE}===== ${NC}"
12
13 if ! command -v distrobox && /dev/null; then
14   echo -e "${RED}Error: distrobox no esta instalado.${NC}"
15   exit 1
16 fi
17
18 echo -e "${BLUE}[1/4] Verificando contenedor Distrobox...${NC}"
19 if ! distrobox list | grep -q "ros2-humble"; then
20   distrobox create -n ros2-humble -i ubuntu:22.04 -y
21 fi
22 echo -e "${GREEN}[OK] Contenedor 'ros2-humble' detectado.${NC}"
23
24 echo -e "${BLUE}[2/4] Compilando Workspace de ROS2...${NC}"
25 SCRIPT_DIR="$( cd "${DIRNAME}" "${BASH_SOURCE[0]}" && /dev/null && pwd )"
26
27 distrobox enter ros2-humble -- bash -c "
28   cd "${SCRIPT_DIR}/ros2_ws" && \
29   source /opt/ros/humble/setup.bash && \
30   colcon build --packages-select arm_simulation
31 "
32 echo -e "${GREEN}[OK] Workspace compilado exitosamente.${NC}"
33
34 echo -e "${BLUE}[3/4] Configurando permisos de ejecucion...${NC}"
35 chmod +x "${SCRIPT_DIR}/ros2_ws/src/arm_simulation/scripts/trajectory_generator.py"
36 echo -e "${GREEN}[OK] Permisos configurados.${NC}"
37
38 echo -e "${BLUE}[4/4] Generando scripts de ejecucion rapida...${NC}"
39 # Generacion automatica de launch_gazebo.sh, run_trajectory.sh y start_ui.sh ...

```

Listing 6. Script de inicialización y compilación del entorno (setup.sh).

APÉNDICE G

LANZADORES RÁPIDOS DE LA SIMULACIÓN Y DE LA INTERFAZ WEB

```
1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Iniciando Gazebo Classic (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5   export PATH=$(echo \$PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6   source /opt/ros/humble/setup.bash
7   source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8   ros2 launch arm_simulation gazebo.launch.py
9 "
```

Listing 7. Script lanzador de Gazebo Classic (launch_gazebo.sh).

```
1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Ejecutando generador de trayectoria (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5   export PATH=$(echo \$PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6   source /opt/ros/humble/setup.bash
7   source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8   /usr/bin/python3 '$SCRIPT_DIR/ros2_ws/src/arm_simulation/scripts/
9     trajectory_generator.py'
10 "
```

Listing 8. Script ejecutor de la trayectoria cosenoidal (run_trajectory.sh).

```
1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Iniciando servidor web y puente ROS 2 para KiraOne en http://localhost
4   :8080..."
5 if command -v xdg-open && /dev/null; then
6   (sleep 1.5 && xdg-open "http://localhost:8080") &
7 elif command -v sensible-browser && /dev/null; then
8   (sleep 1.5 && sensible-browser "http://localhost:8080") &
9 fi
10
11 distrobox enter ros2-humble -- bash -c "
12   export PATH=$(echo \$PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
13   source /opt/ros/humble/setup.bash
14   source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
15   python3 '$SCRIPT_DIR/ui/web_bridge.py'
16 "
```

Listing 9. Script de inicio del servidor y de la interfaz de control (start_ui.sh).

APÉNDICE H

CÓDIGO DEL FIRMWARE DE CONTROL DE BAJO NIVEL Y LÁSER EN EL ESP32 (MAIN.CPP)

```
1 #include <Arduino.h>
2 #include <ESP32Servo.h>
3 #include <math.h>
4
5 #define SERVO_BASE_PIN 27
6 #define SERVO_SHOULDER_PIN 14
7 #define SERVO_ELBOW_PIN 21
8 #define LASER_PIN 22
9
10 // Constantes fisicas del brazo (metros)
11 const float L0 = 0.150f;
12 const float L1 = 0.200f;
13 const float L2 = 0.200f;
14
15 // Limites de las articulaciones (radianes)
16 const float MIN_THETA1 = -1.5708f; // -90 grados
17 const float MAX_THETA1 = 1.5708f; // +90 grados
18 const float MIN_THETA2 = -0.1745f; // -10 grados
19 const float MAX_THETA2 = 1.5708f; // +90 grados
20 const float MIN_THETA3 = -2.0944f; // -120 grados
21 const float MAX_THETA3 = 0.1745f; // +10 grados
22
23 const unsigned long SERIAL_TIMEOUT_MS = 1000;
24
25 Servo servoBase;
26 Servo servoShoulder;
27 Servo servoElbow;
28
29 // Variables de estado (microsegundos correspondientes a la posicion del servo)
30 int currentBaseUs = 1500;
31 int currentShoulderUs = 1500;
32 int currentElbowUs = 1500;
33
34 unsigned long lastPacketTime = 0;
35 bool isEmergency = false;
36
37 /*
38  * Convierte un angulo articular en radianes al correspondiente ancho de pulso PWM
39  * en microsegundos,
40  * limitando el resultado entre los margenes fisicos y de seguridad del servomotor.
41  */
42 int angleToMicroseconds(float angle_rad, float min_rad, float max_rad) {
43   float clamped = constrain(angle_rad, min_rad, max_rad);
44   float us = 1500.0f + clamped * (2000.0f / PI);
45   return constrain((int)us, 500, 2500);
46 }
```

```
46
47 /*
48  * Resuelve analiticamente el modelo cinematico inverso para un brazo de 3 GDL bajo
49  * la configuracion geometrica de codo arriba a partir de una posicion cartesiana (
50   x, y, z).
51 */
52 bool solveInverseKinematics(float x, float y, float z, float &t1, float &t2, float
53   &t3) {
54   t1 = atan2(y, x);
55
56   float r = sqrt(x * x + y * y);
57   float z_prime = z - L0;
58
59   float psi_numerator = x * x + y * y + z_prime * z_prime - L1 * L1 - L2 * L2;
60   float psi_denominator = 2.0f * L1 * L2;
61   float psi = psi_numerator / psi_denominator;
62
63   if (psi < -1.0f || psi > 1.0f) {
64     return false;
65   }
66
67   t3 = atan2(-sqrt(1.0f - psi * psi), psi);
68
69   float k1 = L1 + L2 * cos(t3);
70   float k2 = L2 * sin(t3);
71   t2 = atan2(k1 * z_prime - k2 * r, k1 * r + k2 * z_prime);
72   return true;
73 }
74
75 /*
76  * Activa de forma inmediata el estado de emergencia, apaga el emisor laser de
77  * corte
78  * y desacopla la senal de los servomotores para liberar la tension de la
79  * estructura.
80 */
81 void triggerEmergency(const char* reason) {
82   isEmergency = true;
83   digitalWrite(LASER_PIN, LOW);
84
85   servoBase.detach();
86   servoShoulder.detach();
87   servoElbow.detach();
88
89   Serial.print("\n=== PARADA DE EMERGENCIA ACTIVA ===\nMotivo: ");
90   Serial.println(reason);
91   Serial.println("Reinicie el ESP32 para reestablecer el sistema.");
92 }
93
94 /*
95  * Mueve todos los servos gradualmente en pasos simultaneos para evitar picos de
96  * corriente
97  * bruscos que provoquen el apagado o proteccion de la fuente de alimentacion ATX.
98 */
99 void moveToTarget(int targetBaseUs, int targetShoulderUs, int targetElbowUs, int
100   stepUs = 15, int stepDelay = 15) {
101   bool baseMoving = true;
102   bool shoulderMoving = true;
103   bool elbowMoving = true;
104
105   while (baseMoving || shoulderMoving || elbowMoving) {
106     if (isEmergency) {
107       return;
108     }
109
110     // Alimentar el temporizador del watchdog de seguridad durante el recorrido
111     lastPacketTime = millis();
112
113     baseMoving = (currentBaseUs != targetBaseUs);
114     shoulderMoving = (currentShoulderUs != targetShoulderUs);
115     elbowMoving = (currentElbowUs != targetElbowUs);
116
117     if (baseMoving) {
118       if (abs(targetBaseUs - currentBaseUs) <= stepUs) {
119         currentBaseUs = targetBaseUs;
120       } else {
121         currentBaseUs += (targetBaseUs > currentBaseUs) ? stepUs : -stepUs;
122       }
123       servoBase.writeMicroseconds(currentBaseUs);
124     }
125
126     if (shoulderMoving) {
127       if (abs(targetShoulderUs - currentShoulderUs) <= stepUs) {
128         currentShoulderUs = targetShoulderUs;
129       } else {
130         currentShoulderUs += (targetShoulderUs > currentShoulderUs) ? stepUs : -
131         stepUs;
132       }
133       servoShoulder.writeMicroseconds(currentShoulderUs);
134     }
135
136     if (elbowMoving) {
137       if (abs(targetElbowUs - currentElbowUs) <= stepUs) {
138         currentElbowUs = targetElbowUs;
139       } else {
140         currentElbowUs += (targetElbowUs > currentElbowUs) ? stepUs : -stepUs;
141       }
142       servoElbow.writeMicroseconds(currentElbowUs);
143     }
144
145     if (baseMoving || shoulderMoving || elbowMoving) {
146       delay(stepDelay);
147     }
148   }
149 }
```

```
145 void setup() {
146   Serial.begin(115200);
147   delay(500);
148   Serial.println("\n=== ESP32 3-GDL Laser Arm Controller (Gradual Move) ===");
149 }
```

```

150 pinMode(LASER_PIN, OUTPUT);
151 digitalWrite(LASER_PIN, LOW);
152
153 ESP32PWM::allocateTimer(0);
154 ESP32PWM::allocateTimer(1);
155 ESP32PWM::allocateTimer(2);
156
157 servoBase.setPeriodHertz(50);
158 servoShoulder.setPeriodHertz(50);
159 servoElbow.setPeriodHertz(50);
160
161 // Inicializar servos y asociar pines
162 servoBase.attach(SERVO_BASE_PIN, 500, 2500);
163 servoShoulder.attach(SERVO_SHOULDER_PIN, 500, 2500);
164 servoElbow.attach(SERVO_ELBOW_PIN, 500, 2500);
165
166 // Establecer posicion central por defecto al encender
167 servoBase.writeMicroseconds(currentBaseUs);
168 servoShoulder.writeMicroseconds(currentShoulderUs);
169 servoElbow.writeMicroseconds(currentElbowUs);
170
171 lastPacketTime = millis();
172 Serial.println("Sistema listo. Esperando comandos seriales...");
173 }
174
175 void loop() {
176   if (isEmergency) {
177     digitalWrite(LASER_PIN, LOW);
178     delay(100);
179     return;
180   }
181
182   // Watchdog de seguridad: apaga el laser si se pierde la comunicacion serial
183   if (millis() - lastPacketTime > SERIAL_TIMEOUT_MS) {
184     digitalWrite(LASER_PIN, LOW);
185   }
186
187   if (Serial.available() > 0) {
188     String line = Serial.readStringUntil('\n');
189     line.trim();
190
191     if (line.length() > 0) {
192       // Comando manual para parada de emergencia
193       if (line.equalsIgnoreCase("ESTOP") || line.equalsIgnoreCase("EMERGENCY")) {
194         triggerEmergency("Comando manual recibido");
195         return;
196       }
197
198       int idxX = line.indexOf("X:");
199       int idxY = line.indexOf("Y:");
200       int idxZ = line.indexOf("Z:");
201
202       if (idxX != -1 && idxY != -1 && idxZ != -1) {
203         int endX = line.indexOf(',', idxX);
204         int endY = line.indexOf(',', idxY);
205         int endZ = line.length();
206
207         String xStr = line.substring(idxX + 2, endX != -1 ? endX : line.length());
208         String yStr = line.substring(idxY + 2, endY != -1 ? endY : line.length());
209         String zStr = line.substring(idxZ + 2, endZ);
210
211         float raw_x = xStr.toFloat();
212         float raw_y = yStr.toFloat();
213         float raw_z = zStr.toFloat();
214
215         float scale = 1.0f;
216         if (abs(raw_x) > 2.0f || abs(raw_y) > 2.0f || abs(raw_z) > 2.0f) {
217           scale = 0.001f;
218         }
219
220         float x = raw_x * scale;
221         float y = raw_y * scale;
222         float z = raw_z * scale;
223
224         float thetal = 0.0f, theta2 = 0.0f, theta3 = 0.0f;
225         if (solveInverseKinematics(x, y, z, thetal, theta2, theta3)) {
226           lastPacketTime = millis();
227           digitalWrite(LASER_PIN, HIGH);
228
229           int base_us = angleToMicroseconds(thetal, MIN_THETAL, MAX_THETAL);
230           int shoulder_us = angleToMicroseconds(theta2, MIN_THETA2, MAX_THETA2);
231           int elbow_us = angleToMicroseconds(theta3, MIN_THETA3, MAX_THETA3);
232
233           // Movimiento de servos en pasos graduales suaves
234           moveToTarget(base_us, shoulder_us, elbow_us);
235
236           Serial.print("OK | Target: X=");
237           Serial.print(x, 4);
238           Serial.print(" Y=");
239           Serial.print(y, 4);
240           Serial.print(" Z=");
241           Serial.print(z, 4);
242           Serial.print(" | Joints: T1=");
243           Serial.print(thetal * 180.0f / PI, 1);
244           Serial.print(" T2=");
245           Serial.print(theta2 * 180.0f / PI, 1);
246           Serial.print(" T3=");
247           Serial.println(theta3 * 180.0f / PI, 1);
248         } else {
249           Serial.println("ERR: Posicion cartesiana fuera del area de alcance.");
250         }
251       } else {
252         Serial.println("WARN: Formato invalido. Use 'X:val,Y:val,Z:val'");
253       }
254     }
255   }
256 }

```

Listing 10. Firmware embebido en C++ para control de servomotores y conmutación láser en el ESP32 (main.cpp).

APÉNDICE I CÓDIGO DEL ALGORITMO DE ADELGAZAMIENTO ZHANG-SUEN (WEB_BRIDGE.PY)

```

1 def zhang_suen_thinning(img):
2     import numpy as np
3     im = (img > 0).astype(np.uint8)
4     im = np.pad(im, 1, mode='constant', constant_values=0)
5
6     while True:
7         # Paso 1
8         p2 = im[:-2, 1:-1]
9         p3 = im[:-2, 2:]
10        p4 = im[1:-1, 2:]
11        p5 = im[2:, 2:]
12        p6 = im[2:, 1:-1]
13        p7 = im[2:, :-2]
14        p8 = im[1:-1, :-2]
15        p9 = im[:-2, :-2]
16
17        B = p2 + p3 + p4 + p5 + p6 + p7 + p8 + p9
18
19        A = ((p2 == 0) & (p3 == 1)).astype(np.uint8) + \
20            ((p3 == 0) & (p4 == 1)).astype(np.uint8) + \
21            ((p4 == 0) & (p5 == 1)).astype(np.uint8) + \
22            ((p5 == 0) & (p6 == 1)).astype(np.uint8) + \
23            ((p6 == 0) & (p7 == 1)).astype(np.uint8) + \
24            ((p7 == 0) & (p8 == 1)).astype(np.uint8) + \
25            ((p8 == 0) & (p9 == 1)).astype(np.uint8) + \
26            ((p9 == 0) & (p2 == 1)).astype(np.uint8)
27
28        cond1 = (B >= 2) & (B <= 6) & (A == 1) & (p2 * p4 * p6 == 0) & (p4 * p6 *
29            p8 == 0)
30        inner = im[1:-1, 1:-1]
31        to_delete_1 = (inner == 1) & cond1
32        im[1:-1, 1:-1] = np.where(to_delete_1, 0, inner)
33
34        # Paso 2
35        p2 = im[:-2, 1:-1]
36        p3 = im[:-2, 2:]
37        p4 = im[1:-1, 2:]
38        p5 = im[2:, 2:]
39        p6 = im[2:, 1:-1]
40        p7 = im[2:, :-2]
41        p8 = im[1:-1, :-2]
42        p9 = im[:-2, :-2]
43
44        B = p2 + p3 + p4 + p5 + p6 + p7 + p8 + p9
45        A = ((p2 == 0) & (p3 == 1)).astype(np.uint8) + \
46            ((p3 == 0) & (p4 == 1)).astype(np.uint8) + \
47            ((p4 == 0) & (p5 == 1)).astype(np.uint8) + \
48            ((p5 == 0) & (p6 == 1)).astype(np.uint8) + \
49            ((p6 == 0) & (p7 == 1)).astype(np.uint8) + \
50            ((p7 == 0) & (p8 == 1)).astype(np.uint8) + \
51            ((p8 == 0) & (p9 == 1)).astype(np.uint8) + \
52            ((p9 == 0) & (p2 == 1)).astype(np.uint8)
53
54        cond2 = (B >= 2) & (B <= 6) & (A == 1) & (p2 * p4 * p8 == 0) & (p2 * p6 *
55            p8 == 0)
56        inner = im[1:-1, 1:-1]
57        to_delete_2 = (inner == 1) & cond2
58        im[1:-1, 1:-1] = np.where(to_delete_2, 0, inner)
59
60        if not np.any(to_delete_1) and not np.any(to_delete_2):
61            break
62
63        return (im[1:-1, 1:-1] * 255).astype(np.uint8)

```

Listing 11. Algoritmo de esqueletización vectorizada de Zhang-Suen (web_bridge.py).

APÉNDICE J CÓDIGO DE VERIFICACIÓN CINEMÁTICA Y DH (TEST_KINEMATICS.PY)

```

1 import numpy as np
2
3 # Constantes del robot
4 d1 = 60.0
5 d2 = 36.173
6 a2 = 14.915
7 a3 = 146.190
8 a4 = 146.823
9 d4 = -4.0
10
11 phi2 = 1.030041
12 phi3 = -0.191392
13
14 def dh_matrix(theta, d, a, alpha):
15     alpha_rad = np.radians(alpha) if isinstance(alpha, (int, float)) else alpha
16     return np.array([
17         [np.cos(theta), -np.sin(theta)*np.cos(alpha_rad), np.sin(theta)*np.sin(
18             alpha_rad), a*np.cos(theta)],
19         [np.sin(theta), np.cos(theta)*np.cos(alpha_rad), -np.cos(theta)*np.sin(
20             alpha_rad), a*np.sin(theta)],
21         [0, np.sin(alpha_rad), np.cos(alpha_rad),
22             d],
23         [0, 1, 0, 0],
24     ])
25
26 def fk_dh(thetal, theta2, theta3):

```

```

24     theta2_star = theta2 + phi2
25     theta3_star = theta3 + phi3 - phi2
26
27     T1 = dh_matrix(theta1, d1 + d2, a2, 90)
28     T2 = dh_matrix(theta2_star, 0, a3, 0)
29     T3 = dh_matrix(theta3_star, d4, a4, 0)
30
31     T = T1 @ T2 @ T3
32     return T
33
34 def fk_analytical(theta1, theta2, theta3):
35     theta2_star = theta2 + phi2
36     theta3_star = theta3 + phi3 - phi2
37
38     x_plane = a2 + a3 * np.cos(theta2_star) + a4 * np.cos(theta2_star + theta3_star
39 )
40     z_plane = d1 + d2 + a3 * np.sin(theta2_star) + a4 * np.sin(theta2_star +
41 theta3_star)
42     x = x_plane * np.cos(theta1) + d4 * np.sin(theta1)
43     y = x_plane * np.sin(theta1) - d4 * np.cos(theta1)
44     z = z_plane
45     return np.array([x, y, z])
46
47 # Probar con angulos cero
48 t1, t2, t3 = 0.0, 0.0, 0.0
49 T_dh = fk_dh(t1, t2, t3)
50 pos_analyt = fk_analytical(t1, t2, t3)
51
52 print("D-H End Effector Position (3 matrices):")
53 print(pos_dh)
54 print("\nAnalytical End Effector Position:")
55 print(pos_analyt)

```

Listing 12. Comparación de modelos de cinemática directa y Denavit-Hartenberg (test_kinematics.py).

APÉNDICE K

CÓDIGO DE VALIDACIÓN DE ALINEACIÓN DE MARCOS TF (TEST_TF_ALIGNMENT.PY)

```

1  #!/usr/bin/env python3
2  import rclpy
3  from rclpy.node import Node
4  import tf2_ros
5  import math
6  import numpy as np
7
8  class TFAlignmentValidator(Node):
9      def __init__(self):
10         super().__init__('tf_alignment_validator')
11         self.tf_buffer = tf2_ros.Buffer()
12         self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
13
14         # Esperar un poco a que se llene el buffer de TFs
15         self.timer = self.create_timer(1.0, self.validate_alignment)
16         self.get_logger().info("Validator node started. Waiting for TF data...")
17
18     def validate_alignment(self):
19         self.timer.cancel()
20
21         frames = ['system1', 'system2', 'system3', 'flange']
22         base_frame = 'link1'
23
24         print("\n" + "="*70)
25         print("VALIDACION DE ALINEACION DE EJES X (VISTA LATERAL / PLANO X-Z de link1)")
26         print("="*70)
27
28         all_parallel = True
29
30         for frame in frames:
31             try:
32                 # Obtener la transformacion de link1 al sistema
33                 now = rclpy.time.Time()
34                 trans = self.tf_buffer.lookup_transform(base_frame, frame, now,
35 timeout=rclpy.duration.Duration(seconds=2.0))
36
37                 # Obtener cuaternion
38                 q = trans.transform.rotation
39
40                 # Convertir cuaternion a matriz de rotacion
41                 R = self.quaternion_to_matrix(q.x, q.y, q.z, q.w)
42
43                 # El eje X del frame en coordenadas de link1 es la primera columna
44                 x_axis = R[:, 0]
45
46                 # Calcular angulo en el plano vertical X-Z de link1
47                 angle_rad = math.atan2(x_axis[2], x_axis[0])
48                 angle_deg = math.degrees(angle_rad)
49
50                 print(f"\nFrame: {frame}")
51                 print(f" Vector eje X (en frame link1): [{x_axis[0]:.6f}, {x_axis[1]:.6f}, {x_axis[2]:.6f}]")
52                 print(f" Desviacion lateral (Y): {x_axis[1]:.6f} (debe ser aprox 0)")
53                 print(f" Angulo en plano X-Z (Pitch): {angle_deg:.4f} deg")
54
55                 if abs(x_axis[1]) > 0.005:
56                     print(" [WARN] ADVERTENCIA: El eje X no esta en el mismo plano de movimiento (Y != 0)")
57                     all_parallel = False
58                     elif abs(angle_deg) > 0.1:

```

```

58         print(f" [WARN] ADVERTENCIA: El eje X no es paralelo al eje X
59 horizontal (Angulo = {angle_deg:.4f} deg)")
60         all_parallel = False
61     except:
62         print(" [OK] ALINEACION PERFECTA (eje X paralelo y horizontal)")
63
64     except Exception as e:
65         self.get_logger().error(f"No se pudo obtener la transformacion para {frame}: {str(e)}")
66         all_parallel = False
67
68     print("\n" + "="*70)
69     if all_parallel:
70         print(" RESULTADO FINAL: [OK] TODOS LOS EJES X SON PARALELOS Y HORIZONTALES!")
71     else:
72         print(" El brazo esta perfectamente recto y alineado en la pose cero de la simulacion.")
73     else:
74         print(" RESULTADO FINAL: [ERR] EXISTEN DESALINEACIONES O DESVIACIONES FUERA DE LIMITES.")
75     print("="*70 + "\n")
76
77     rclpy.shutdown()
78
79 def quaternion_to_matrix(self, x, y, z, w):
80     return np.array([
81         [1 - 2*(y**2 + z**2), 2*(x*y - w*z), 2*(x*z + w*y)],
82         [2*(x*y + w*z), 1 - 2*(x**2 + z**2), 2*(y*z - w*x)],
83         [2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x**2 + y**2)]
84     ])
85
86 def main(args=None):
87     rclpy.init(args=args)
88     node = TFAlignmentValidator()
89     try:
90         rclpy.spin(node)
91     except SystemExit:
92         pass
93
94 if __name__ == '__main__':
95     main()

```

Listing 13. Validación de paralelismo de ejes longitudinales en base al árbol de transformadas TF (test_tf_alignment.py).

APÉNDICE L

CÓDIGO DE VERIFICACIÓN ANALÍTICA DE URDF (TEST_URDF_TF_ALIGNMENT.PY)

```

1  #!/usr/bin/env python3
2  import math
3  import numpy as np
4
5  def rpy_to_matrix(r, p, y):
6      cr, sr = math.cos(r), math.sin(r)
7      cp, sp = math.cos(p), math.sin(p)
8      cy, sy = math.cos(y), math.sin(y)
9
10     Rx = np.array([[1, 0, 0], [0, cr, -sr], [0, sr, cr]])
11     Ry = np.array([[cp, 0, sp], [0, 1, 0], [-sp, 0, cp]])
12     Rz = np.array([[cy, -sy, 0], [sy, cy, 0], [0, 0, 1]])
13
14     return Rz @ Ry @ Rx
15
16 # Datos de origen de las articulaciones de la simulacion (URDF actual)
17 R_base = rpy_to_matrix(1.57079632679, 0, 0)
18 R_base_l1 = rpy_to_matrix(-1.57079632679, 0, 0)
19 R_l1_l2 = rpy_to_matrix(1.57079632679, 0, 0)
20 R_l2_l3 = rpy_to_matrix(0, 0, 0)
21 R_l3_flange = rpy_to_matrix(0, 0, 0)
22
23 # Marcos de referencia visuales (visualize_frame) en el URDF actual
24 R_base_sys0 = rpy_to_matrix(-1.57079632679, 0, 0)
25 R_l2_sys1 = rpy_to_matrix(0, 0, 0)
26 R_l3_sys2 = rpy_to_matrix(0, 0, 0)
27 R_flange_sys3 = rpy_to_matrix(0, 0, 0)
28
29 transforms = {
30     "system1": R_l1_l2 @ R_l2_sys1,
31     "system2": R_l1_l2 @ R_l2_l3 @ R_l3_sys2,
32     "system3": R_l1_l2 @ R_l2_l3 @ R_l3_flange @ R_flange_sys3,
33     "flange": R_l1_l2 @ R_l2_l3 @ R_l3_flange
34 }
35
36 print("\n" + "="*75)
37 print(" CALCULO ANALITICO DE ALINEACION DE EJES X (DESDE LA SIMULACION URDF)")
38 print(" Plano de referencia: vista lateral (plano X-Z de link1)")
39 print("="*75)
40
41 all_parallel = True
42
43 for name, R_l1_S in transforms.items():
44     x_axis_in_l1 = R_l1_S[:, 0]
45     y_deviation = x_axis_in_l1[1]
46     angle_rad = math.atan2(x_axis_in_l1[2], x_axis_in_l1[0])
47     angle_deg = math.degrees(angle_rad)
48
49     print(f"\nSistema: {name}")
50     print(f" Direccion del eje X en link1: [{x_axis_in_l1[0]:.6f}, {x_axis_in_l1[1]:.6f}, {x_axis_in_l1[2]:.6f}]")
51     print(f" Desviacion lateral (Y): {y_deviation:.6f} (debe ser 0 para estar coplanar)")
52     print(f" Angulo lateral (Pitch): {angle_deg:.4f} deg (debe ser 0 deg para estar horizontal)")
53

```



```

54 if abs(y_deviation) > 1e-5:
55     print(" [ERR] ERROR: El eje X se sale del plano vertical (desviacion Y no
        es cero).")
56     all_parallel = False
57 elif abs(angle_deg) > 1e-5:
58     print(f" [ERR] ERROR: El eje X no es horizontal en la vista lateral (
        Angulo = {angle_deg:.4f} deg).")
59     all_parallel = False
60 else:
61     print(" [OK] PERFECTO: El eje X es coplanar, paralelo y horizontal.")
62
63 print("\n" + "="*75)
64 if all_parallel:
65     print(" VERIFICACION EXITOSA: [OK] TODOS LOS EJES X SON PARALELOS Y
        PERFECTAMENTE ALINEADOS!")
66 else:
67     print(" VERIFICACION FALLIDA: [ERR] Existen desalineaciones entre los marcos.")
68 print("="*75 + "\n")

```

Listing 14. Verificación analítica de la rotación y coplanaridad de las articulaciones en el modelo URDF (test_urdf_tf_alignment.py).

APÉNDICE M

CÓDIGO DE LECTURA DE DISTANCIAS DE MARCOS TF EN GAZEBO (GET_TF_DISTANCES.PY)

```

1 #!/usr/bin/env python3
2 import rclpy
3 from rclpy.node import Node
4 import tf2_ros
5 import sys
6
7 class TFEchoAll(Node):
8     def __init__(self):
9         super().__init__('tf_echo_all')
10        self.tf_buffer = tf2_ros.Buffer()
11        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
12        self.timer = self.create_timer(1.0, self.query_all_tfs)
13        self.run_once = False
14
15    def query_all_tfs(self):
16        if self.run_once:
17            return
18
19        frames = ['link1', 'link2', 'link3', 'flange']
20        base = 'base_link'
21
22        print("\n=== COORDENADAS DE LAS ARTICULACIONES DESDE TF EN GAZEBO ===")
23        print(f"Sistema de referencia base: {base}\n")
24
25        success = True
26        for frame in frames:
27            try:
28                trans = self.tf_buffer.lookup_transform(base, frame, rclpy.time.
29                Time())
30                translation = trans.transform.translation
31                rotation = trans.transform.rotation
32                x, y, z, w = rotation.x, rotation.y, rotation.z, rotation.w
33
34                print(f"-> {frame} respecto a {base}:")
35                print(f"  Traslacion: [{translation.x:.6f}, {translation.y:.6f}, {
36                translation.z:.6f}] m")
37                print(f"    [{translation.x*1000:.3f}, {translation.y
38                *1000:.3f}, {translation.z*1000:.3f}] mm")
39                print(f"  Rotacion (Quaternion): xyzw = [{x:.6f}, {y:.6f}, {z:.6f
40                }, {w:.6f}]\n")
41
42            except Exception as e:
43                print(f"Error buscando transformada de {base} a {frame}: {str(e)}")
44                success = False
45
46        if success:
47            self.run_once = True
48            sys.exit(0)
49
50    def main(args=None):
51        rclpy.init(args=args)
52        node = TFEchoAll()
53        try:
54            rclpy.spin(node)
55        except SystemExit:
56            pass
57        except KeyboardInterrupt:
58            pass
59        finally:
60            node.destroy_node()
61            rclpy.shutdown()
62
63 if __name__ == '__main__':
64     main()

```

Listing 15. Script de lectura y visualización de transformadas de articulaciones desde TF en Gazebo (get_tf_distances.py).

APÉNDICE N

CÓDIGO DEL PARSER Y ANALIZADOR DE PARÁMETROS XACRO (GET_GAZEBO_DISTANCES.PY)

```

1 import xml.etree.ElementTree as ET
2 import math
3 import os
4
5 def parse_urdf():
6     urdf_path = "/home/reve/Documents/UC/robotica/finalProject/ros2_ws/src/
        arm_simulation/urdf/arm.urdf.xacro"
7     if not os.path.exists(urdf_path):
8         print(f"Error: No se encontro el archivo URDF en {urdf_path}")
9         return
10
11     tree = ET.parse(urdf_path)
12     root = tree.getroot()
13
14     print("=== DISTANCIAS Y LONGITUDES EXTRAIDAS DEL MODELO URDF EN GAZEBO ===\n")
15
16     joint_world = root.find(".//joint[@name='world_to_base']")
17     if joint_world is not None:
18         origin = joint_world.find("origin")
19         print(f"world_to_base:")
20         xyz = {origin.attrib.get('xyz')}
21         rpy = {origin.attrib.get('rpy')}\n")
22
23     joint1 = root.find(".//joint[@name='joint1']")
24     if joint1 is not None:
25         origin = joint1.find("origin")
26         print(f"joint1 (Base a Link1):")
27         xyz = {origin.attrib.get('xyz')} (Desfase vertical base height d1
28         = 60 mm)
29         rpy = {origin.attrib.get('rpy')}\n")
30
31     joint2 = root.find(".//joint[@name='joint2']")
32     if joint2 is not None:
33         origin = joint2.find("origin")
34         xyz = [float(x) for x in origin.attrib.get('xyz').split()]
35         print(f"joint2 (Link1 a Hombro):")
36         xyz = {origin.attrib.get('xyz')} (Desfase radial a2 = {xyz
37         [0]*1000:.3f} mm, Desfase vertical d2 = {xyz[2]*1000:.3f} mm)
38         rpy = {origin.attrib.get('rpy')}\n")
39
40     joint3 = root.find(".//joint[@name='joint3']")
41     if joint3 is not None:
42         origin = joint3.find("origin")
43         xyz = [float(x) for x in origin.attrib.get('xyz').split()]
44         length = math.sqrt(xyz[0]**2 + xyz[1]**2) * 1000
45         angle = math.atan2(xyz[1], xyz[0]) * 180 / math.pi
46         print(f"joint3 (Hombro a Codo):")
47         xyz = {origin.attrib.get('xyz')}
48         print(f"  Longitud del brazo (Link2) a3: {length:.3f} mm")
49         print(f"  Angulo de inclinacion phi2: {angle:.6f} deg ({math.atan2(xyz[1],
50         xyz[0]):.6f} rad)\n")
51
52     joint_flange = root.find(".//joint[@name='joint_3-flange']")
53     if joint_flange is not None:
54         origin = joint_flange.find("origin")
55         xyz = [float(x) for x in origin.attrib.get('xyz').split()]
56         length = math.sqrt(xyz[0]**2 + xyz[1]**2) * 1000
57         angle = math.atan2(xyz[1], xyz[0]) * 180 / math.pi
58         print(f"joint_3-flange (Codo a Breda):")
59         xyz = {origin.attrib.get('xyz')}
60         print(f"  Longitud del antebrazo (Link3) a4: {length:.3f} mm")
61         print(f"  Angulo de inclinacion phi3: {angle:.6f} deg ({math.atan2(xyz[1],
62         xyz[0]):.6f} rad)\n")
63         print(f"  Desfase transversal d4: {xyz[2]*1000:.3f} mm\n")
64
65     if __name__ == '__main__':
66         parse_urdf()

```

Listing 16. Script parser XML para extraer dimensiones y ángulos del archivo URDF/Xacro (get_gazebo_distances.py).

REFERENCIAS

- [1] J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2017.
- [2] I. Hinojosa, "Repositorio de Código del Proyecto FP_Robotics", GitHub, 2026. Disponible en: https://github.com/Reve7339/FP_Robotics.